一 前期设置

1.IP设置

确保工控机ip为雷达数据和机械臂数据的目的ip，默认设置工控机ip为192.168.1.90。

雷达数据的目的ip修改方式为使用电脑端的app修改。

机械臂数据的目的ip修改方式为使用示教显示器上的APP进行修改。

2.生成标签

```
$ source ~/catkint_workspace/devel/setup.bash
$ rosrun ar_track_alvar createMarker 0
```

> 后面的数字表示生成标签的id号，可以根据需要生成不同数字的标签

3.打印标签

打印标签时注意尺寸，默认使用3x3 cm的标签，如果打印尺寸和默认尺寸不同，请在~/your_workspace/src/open_manipulator_perceptions/open_manipulator_ar_markers/launch/agx_ar_pose.launch文件中修改标签尺寸。

```
<arg name="user_marker_size"    default="3"/>
```

> 在使用过程中默认0号标签贴在物体上，1号标签贴在物品摆放平台上。

4.构建地图

```
$ roslaunch agilexpro open_lidar.launch
$ roslaunch agilexpro gmapping.launch
```

构建好地图之后，把地图保存在~/catkin_workspace/src/agilexpro/maps目录下

```
$ rosrun map_server map_saver -f ~/catkin_workspace/src/agilexpro/maps/map
```

> 最后面的map是生成地图的名字，在导航的时候默认是加载以map命名地图文件，如果在命名的时候取了别的名字需要修改加载地图的名字。把~catkin_workspace/src/agilexpro/launch/navigation_4wd.launch文件中下面地图名字改为你命名的名字。
>
> ```
> <node name="map_server" pkg="map_server" type="map_server" args="$(find agilexpro)/maps/map.yaml" output="screen">
> ```

二 两点之间抓取投放演示

> 下面指令执行之前需要source工作空间

1. 启动机械臂

```
$ roslaunch agx_aubo_bringup setup_arm.launch
```

> 开启机械臂控制柜的电源，解锁机械臂每个关节的保险。长按机械臂控制柜上连接的示教显示屏上的电源按钮，在弹出的小窗口点击保存->启动，就开启了机械臂的电气开关。

在这个launch file里面可支持修改一个参数，robot_ip。这里传入的ip为机械臂控制柜默认ip192.168.1.100。

```
<launch>
  <arg name="robot_ip" default="192.168.1.100" />
  <include file="$(find aubo_i5_moveit_config)/launch/moveit_planning_execution.launch">
    <arg name="robot_ip" value="$(arg robot_ip)"/>
  </include>
</launch>
```

2.启动摄像头

```
$ roslaunch agx_aubo_bringup open_camera.launch
```

> 注意启动之前需要用USB3.0的数据线连接摄像头与工控机。

该launch file支持修改的参数有下面几个：

```
    <arg name="camera_namespace" value="cam1"/>
    <arg name="serial_no" value="035422071503"/>
    <arg name="node_name" value="cam1"/>
    <arg name="use_quaternion" value="true"/>
    <!--<arg name="roll" value="0"/>
    <arg name="pitch" value="-1.57"/>
    <arg name="yaw" value="-1.57"/>
    <arg name="x" value="0.05"/>
    <arg name="y" value="0.03"/>
    <arg name="z" value="0.0"/>-->
    <arg name="x" value="0.0446414171704"/>
    <arg name="y" value="0.053996778351"/>
    <arg name="z" value="0.0600140964856"/>
    <arg name="r_x" value="-0.000283027265062"/>
    <arg name="r_y" value="-0.0682970373998"/>
    <arg name="r_z" value="0.984025866108"/>
    <arg name="r_w" value="0.164403556559"/>
```

camera_namespace：摄像头节点的命名空间，当启动多个摄像头时可以需要修改每个摄像头的命名空间。

serial_no：摄像头的序列号S/N码，当需要启动多个摄像头的时候可以传入摄像头的序列号指定打开某个摄像头。

node_name：一般与命名空间相同。

use_quaternion：true表示使用四元数，false表示使用欧拉角

x,y,z：摄像头距离机械臂的位置。

roll,pitch,yaw：摄像头姿态欧拉角。

r_x,r_y,r_z,r_w：摄像头姿态四元数。

在启动这个launch file的时候也启动了图像识别节点，会实时更新物体和物体摆放平台的空间坐标信息，可根据以下接口获取这两点信息。服务名字为/pick_point。服务类型为agx_pick_msg/AgxPickSrv，详细的消息格式如下：

```
int32 q # 0--bottle;1--A;2--B
---
geometry_msgs/Point position
  float64 x
  float64 y
  float64 z
geometry_msgs/Quaternion orientation
  float64 x
  float64 y
  float64 z
  float64 w
```

其中q为0时，返回物体的坐标信息；q为2时，返回物体摆放位置信息。

3.启动机械爪

```
$ roslaunch agx_aubo_bringup open_gripper.launch
```

控制机械爪的服务接口为/pick_state,消息的类型为agx_pick_msg/PickStateSrv ，消息格式如下：

```
int32 moterID  #机械爪的电机个数

int32 pose  #机械爪的指尖位置（mm）

int32 force  #机械爪的抓取力度（N）
---
bool result
```

在dh_gripper_ros功能包里面有个一demo文件可以输入指尖位置来控制机械爪。使用指令如下：

```
$ rosrun dh_hand_driver demo_client
```

4. 启动agx_aubo_pick节点

```
$ roslaunch agx_aubo_bringup agx_aubo_bring.launch
```

该节点需要获取状态机传入的taskid来分别控制机器人执行不同的任务。任务的服务接口为/send_task，服务类型为agx_pick_msg/TaskCmd，消息格式如下：

```
int32 taskID
geometry_msgs/PoseStamped A_goal
  std_msgs/Header header
    uint32 seq
    time stamp
```

```
    string frame_id
  geometry_msgs/Pose pose
    geometry_msgs/Point position
      float64 x
      float64 y
      float64 z
    geometry_msgs/Quaternion orientation
      float64 x
      float64 y
      float64 z
      float64 w
geometry_msgs/PoseStamped B_goal
  std_msgs/Header header
    uint32 seq
    time stamp
    string frame_id
  geometry_msgs/Pose pose
    geometry_msgs/Point position
      float64 x
      float64 y
      float64 z
    geometry_msgs/Quaternion orientation
      float64 x
      float64 y
      float64 z
      float64 w
---
bool result
```

5. 启动smach状态机节点

```
$ rosrun agx_aubo_smach smach_demo.py
```

定义了以下几种状态的转换：

```
    smach.StateMachine.add('WAITFORORDER', WaitFororder(),
                           transitions={'receive':'PUBAGOAL',
                                        'wait':'WAITFORORDER'})

    smach.StateMachine.add('PUBAGOAL', PubAGoal(),
                           transitions={'pubA':'WAITFORREACHGOAL'})

    smach.StateMachine.add('WAITFORREACHGOAL', WaitForReachGoal(),
                           transitions={'no':'WAITFORREACHGOAL',
                                        'reachA':'PICK',
                                        'reachB':'PLACE'})

    smach.StateMachine.add('PICK', Pick(),
                           transitions={'pick':'PUBBGOAL'})

    smach.StateMachine.add('PUBBGOAL', PubBGoal(),
                           transitions={'pubB':'WAITFORREACHGOAL'})

    smach.StateMachine.add('PLACE', Place(),
                           transitions={'place':'WAITFORORDER'})
```

以上是整个系统的状态转换逻辑，可以根据需要添加相应的状态。以第一个状态为例，在这里定义了一个“WAITFORORDER”的状态，这个状态的类为WaitFororder，transitions代表状态的跳转。如果WaitFororder输出的结果为receive则跳转到“PUBAGOAL”，如果状态输出结果为wait，则跳转到“WAITFORORDER”状态。

6. 启动雷达

```
$ roslaunch agilexpro open_lidar.launch
```

7.启动导航

```
$ roslaunch agilexpro navigation_4wd.launch
```

导航的某点话题接口为/move_base_simple/goal，消息类型为geometry_msgs/PoseStamped，消息格式如下：

```
std_msgs/Header header
  uint32 seq
  time stamp
  string frame_id
geometry_msgs/Pose pose
  geometry_msgs/Point position
    float64 x
    float64 y
    float64 z
  geometry_msgs/Quaternion orientation
    float64 x
    float64 y
    float64 z
    float64 w
```

到达某个点之后的反馈信息可以从/move_base/result话题获得，消息类型为base_msgs/MoveBaseActionResult ，消息格式如下：

```
std_msgs/Header header
  uint32 seq
  time stamp
  string frame_id
actionlib_msgs/GoalStatus status
  uint8 PENDING=0
  uint8 ACTIVE=1
  uint8 PREEMPTED=2
  uint8 SUCCEEDED=3
  uint8 ABORTED=4
  uint8 REJECTED=5
  uint8 PREEMPTING=6
  uint8 RECALLING=7
  uint8 RECALLED=8
  uint8 LOST=9
  actionlib_msgs/GoalID goal_id
```

```
    time stamp
    string id
  uint8 status
  string text
move_base_msgs/MoveBaseResult result
```

当机器人到达目标点后可以查看status为3。

8.发送开始任务的指令

```
$ rosrun agx_aubo_pick
```

在/goal_service服务上发送开始执行任务的请求，服务接口为/goal_service，服务类型为agx_pick_msg/OrderSrv消息格式如下，在agx_aubo_pick中有个demo文件可以参考运行。

```
int32 id
geometry_msgs/PoseStamped A_goal
  std_msgs/Header header
    uint32 seq
    time stamp
    string frame_id
  geometry_msgs/Pose pose
    geometry_msgs/Point position
      float64 x
      float64 y
      float64 z
    geometry_msgs/Quaternion orientation
      float64 x
      float64 y
      float64 z
      float64 w
geometry_msgs/PoseStamped B_goal
  std_msgs/Header header
    uint32 seq
    time stamp
    string frame_id
  geometry_msgs/Pose pose
    geometry_msgs/Point position
      float64 x
      float64 y
      float64 z
    geometry_msgs/Quaternion orientation
      float64 x
      float64 y
      float64 z
      float64 w
---
int32 result
```

切换遥控器为指令控制模式，则复合移动机器人开始导航到第一个地点抓取东西。